

Sensor Interfacing

Temperature Sensor:

Transducers convert physical data such as temperature, light intensity, flow and speed to electrical signals.

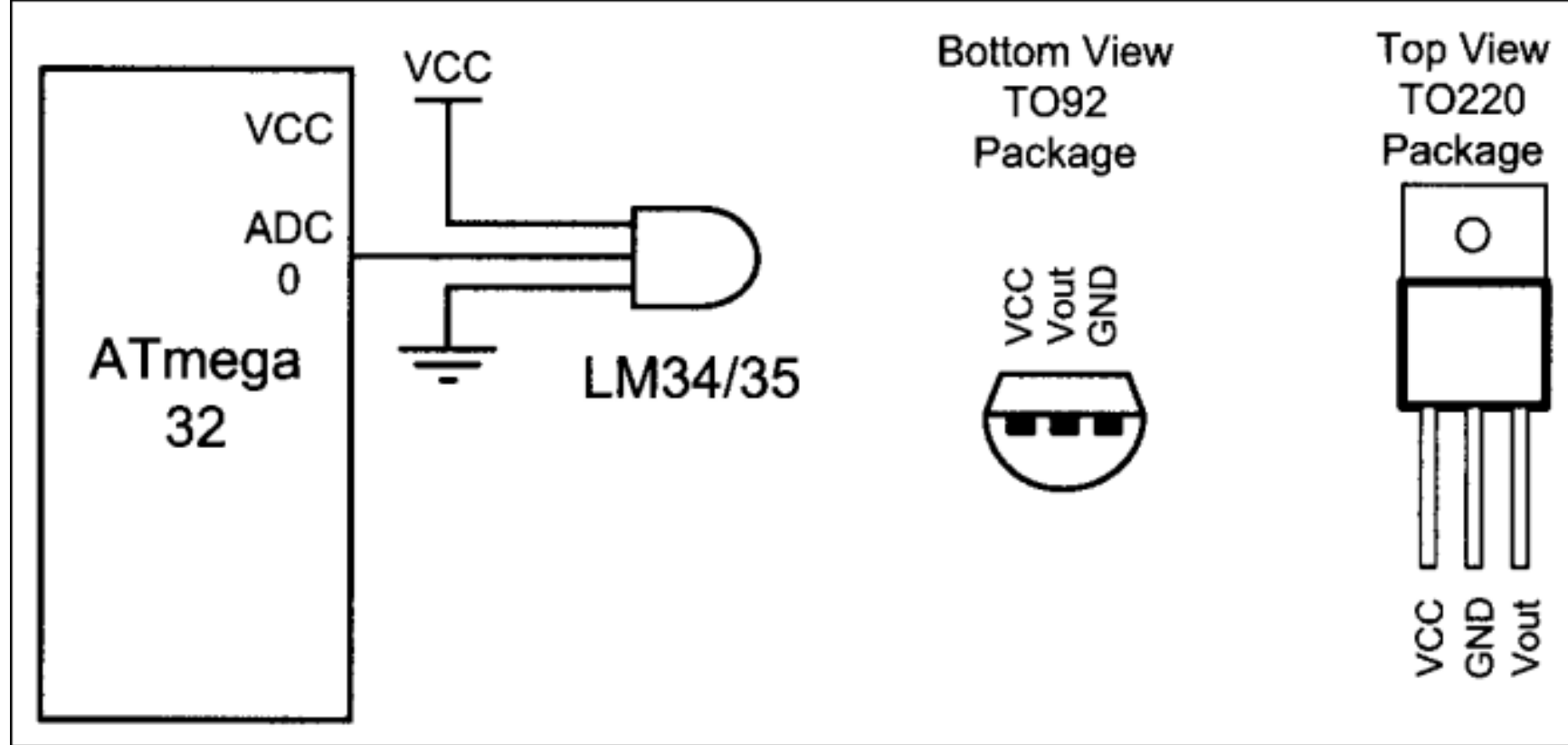
Depending on transducers, the output is produced in the form of voltage, current, resistance, charge, or capacitance.

LM35 sensors are precision integrated-circuit, whose output voltage is linearly proportional to the Celsius. Its output is 10mV for each degree of centigrade Celsius.

Interfacing the LM35

- A/D has 10-bit resolution with a maximum of 1024 steps
- LM35 produces 10mV for each degree of centigrade Celsius
- If we use step size of 10mV, V_{out} will be 10240mV (10.24V)
- The highest output will get for the A/D is 3000mV
- For Internal 2.56V reference voltage, step size is $2.56V/1024 = 2.5mV$
- This is four (4) times of step size 2.5mV to produce 10mV for each degree of temperature
- Scale it by dividing 4 to get the real number for the temperature

Temp. (F)	V _{in} (mV)	# of steps	Binary V _{out} (b9–b0)	Temp. in Binary
0	0	0	00 00000000	00000000
1	10	4	00 00000100	00000001
2	20	8	00 00001000	00000010
3	30	12	00 00001100	00000011
10	100	20	00 00101000	00001010
20	200	80	00 01010000	00010100
30	300	120	00 01111000	00011110
40	400	160	00 10100000	00101000
50	500	200	00 11001000	00110010
60	600	240	00 11110000	00111100
70	700	300	01 00011000	01000110
80	800	320	01 01000000	01010000
90	900	360	01 01101000	01011010
100	1000	400	01 10010000	01100100



Displaying Temperature:

- 10-bit output is divided by 4 to get real temperature
- Chose left-justified option
- Read the ADCH

Write a program to read the sensor and display it on PORTK.

```
#include<avr/io.h>
int main(void)
{
    DDRF &= ~(1<<0); // make PORTF an input for ADC0 input
    DDRK = 0xFF;
    ADMUX = 0xE0;
    ADCSRA = 0x87;
    ADCSRB = 0x00;
    while(1)
    {
        ADCSRA |= (1<<ADSC);
        while(ADCSRA & (1<< ADIF) ==0);
        PORTK = ADCH;
    }
    return 0;
}
```

Write a program to read the sensor and display it on LCD.

```
#include<avr/io.h>
int main(void)
{
    DDRF &= ~(1<<0); // make PORTF an input for ADC0 input
    DDRK = 0xFF;
    ADMUX = 0xE0;
    ADCSRA = 0x87;
    ADCSRB = 0x00;
    while(1)
    {
        ADCSRA |= (1<<ADSC);
        while(ADCSRA & (1<< ADIF) ==0);
        PORTK = ADCH;
    }
    return 0;
}
```